

OOPJ Lecture Day 2

Tue, 08 Sept 26

Recap: Compilation Pipeline

- Five essentials for any C development environment: text editor, compiler, linker, libraries, debugger
- Compiler is responsible for multiple stages, not just error checking:
 - Preprocessing (stage 1): runs before compilation proper
 - Evaluates all preprocessor directives (e.g. `#include`, `#define`)
 - Expands header files in place so declarations are available before syntax checking begins
 - Analogy: like washing and cutting vegetables before cooking; prep must happen before the main process starts
 - Syntax and grammar checking (stage 2): verifies code is syntactically and semantically correct
 - Grammar of a language is also called its semantics
 - Any error here throws a compilation error immediately
 - Code optimization (stage 3): runs alongside conversion to machine code
 - Object code generation: produces machine-dependent `.obj` file, one per `.c` file
 - Four `.c` files → four `.obj` files (`first.obj`, `second.obj`, `third.obj`, `fourth.obj`)
- Header files (`.h`):
 - Contain declarations only, never definitions
 - Cannot be compiled independently; no `.obj` is generated for them
 - Compiled only when included inside a `.c` file
 - A project with hundreds of `.h` files still produces zero `.obj` files from those headers
- Linker responsibilities (runs after compilation):
 - Links all `.obj` files together (e.g. `program.obj` + `calculator.obj`)
 - Resolves calls to predefined functions by attaching library files (e.g. `printf` → `stdio.lib`)
 - Generates the final `.exe` using COFF (Common Object File Format), which is Windows-specific
 - COFF format is platform-dependent: Linux uses a different format, so a Windows `.exe` cannot run on Linux
 - The resulting `.exe` is independently executable on any compatible system
- Modularity example in C:
 - `calculator.c`: contains definitions of `add`, `subtract`, `multiply`, `divide`
 - `calculator.h`: contains declarations (prototypes) of those four functions only
 - `program.c`: contains `main()`, includes `calculator.h`, calls the functions
 - Compiler uses the header to verify function calls match declarations; linker then links `program.obj` and `calculator.obj` together

- This separation is the principle of modularity: main logic separate from functionality

Static vs. Dynamic Link Libraries

- Both SSL (Static Link Library) and DLL (Dynamic Link Library) are binary formats
 - Content is machine-dependent code, like an EXE, but cannot be independently executed
 - They need a carrier: the EXE file
- Static Link Library (SSL / .lib):
 - Attached to the EXE by the linker at link time
 - The entire library becomes part of the EXE binary
 - When the EXE loads into RAM, the full library loads with it, whether used or not
 - Example: if stdio.lib is 1 MB and the program is 10 KB, total RAM usage is ~1.01 MB regardless of how many stdio functions are actually called
 - Extreme case: if a library has 100 functions related to 29th February, they load into RAM every day, not just on 29th February
 - Disadvantage 1: unnecessary memory consumption from unused functions
 - Disadvantage 2: if the library is updated, the EXE must be fully relinked to reflect changes; the new .lib is not automatically picked up
- Dynamic Link Library (DLL / .dll):
 - Not embedded in the EXE; exists as a separate file on disk
 - When the EXE launches, only the EXE loads into RAM; the DLL is not yet loaded
 - DLL is loaded into RAM dynamically at runtime, only when a call is made to a function inside it
 - Example: a 10 MB DLL containing 29th February functions only loads into RAM on 29th February
 - Advantage 1: memory efficient; only loaded when actually needed
 - Advantage 2: changes to the DLL are reflected immediately on the next run of the EXE, no relink required
 - Disadvantage: slight performance cost because loading and linking happen at runtime, not at startup
- Tradeoff summary:
 - SSL preferred when performance is the priority (no runtime loading overhead)
 - DLL preferred when memory optimization or dynamic updates are the priority
- DLL sub-types: implicit linking and explicit linking
 - Implicit linking partially bridges the performance gap between SSL and DLL
- Unnecessary #include statements in .c files cause more libraries to be attached to the EXE, increasing RAM usage at runtime; only include what is actually needed

Pointers Recap

- A pointer is a variable that stores the memory address of another variable
- Primary use case: accessing a variable beyond its scope, but within its lifetime

- Direct access to a variable is only possible within its scope; a pointer allows indirect access from outside that scope
- Example walkthrough:
 - int num1 declared inside main() at memory address 200, holding value 0
 - display() function cannot access num1 directly because num1 is out of scope
 - Solution: pass an int* (integer pointer) as an argument to display()
 - At the call site: pass &num1 (address of num1) to the function
 - Inside display(): the pointer holds address 200; dereferencing it allows reading or modifying num1
- Rule: use pointers when you need to access a variable beyond its scope but the variable is still alive (within its lifetime)
- Students were asked to try this in Visual Studio with the debugger to observe memory addresses directly

OOP: 5 Major Features

Abstraction

- Abstraction is the process of generalization: identifying what is common across multiple real-world entities and extracting those common features into a higher-level parent type
- It is not about hiding unnecessary details (a common misconception); it is about finding and grouping common properties and behaviors
 - If something is truly unnecessary, it would not be included at all
 - The author's phrasing of "hiding unessential features" is misleading; the real meaning is gathering what is common
- Example 1 (vehicles):
 - Sedan, SUV, MUV all share: engine, tyres, steering wheel, gearbox, start/stop/drive behaviors
 - Writing these separately in three classes is repetitive
 - Abstraction: extract the common features into a parent class called PassengerCar
 - PassengerCar generalizes what is common; sedan/SUV/MUV specialize from it
- Example 2 (living things):
 - Man + Woman → Human; Human + Monkey → Ape (Mammal); Mammal + Fish → Living Thing
 - The higher up the hierarchy, the more generalized the type
 - "Living Thing" could be a man, a bird, an ant; "Man" can only be a man
- Abstraction is also called generalization; the two terms are interchangeable in OOP
- Abstract classes (a separate concept in Java/C++) are a specific implementation mechanism; they are not the same as the principle of abstraction itself

Encapsulation and Hiding

- Encapsulation: binding data (properties) and the behaviors (functions) that act upon that data into a single, uniquely identifiable unit
- Example 1: a car — engine, tyres, gearbox, bonnet are all encapsulated together; you do not drive with the engine in one place and the tyres somewhere else
- Example 2: the human body — heart, lungs, kidneys, brain are all encapsulated within one body; you do not forget your kidney at home
- Example 3 (nested encapsulation): room → flat → floor → building → society → street → city → state → country; each is an encapsulated unit within a larger one
- Hiding is a direct consequence of encapsulation:
 - If 10 items are scattered on a table, anyone with access to the table can see them
 - If those 10 items are placed in a locked drawer, only someone with the key can access them
 - Hiding is not possible without encapsulation; you must first encapsulate before you can hide
- Encapsulation and hiding always work at the same level; they are listed together as one of the five major features

Inheritance

- Inheritance is specialization: taking an existing (generalized) type and extending or modifying it to create a new, more specific type
- You do not start from scratch; you inherit all properties and behaviors of the parent and add or change what is needed
- A subclass only makes sense if it adds something new or changes existing behavior; inheriting without modification is pointless
- Real-world analogy: an engineer who can build a car from scratch does not build one to go on a family trip; they buy one. For a rally, they take an existing car and specialize it (higher chassis, extra spare tyres, etc.)
- Four types of inheritance:
 - Single: $A \rightarrow B$ (B inherits from A)
 - Multi-level: $A \rightarrow B \rightarrow C$ (chain of inheritance)
 - Multiple: A and B both inherited into C (C has two parents)
 - Hybrid: combination of multi-level and multiple inheritance
- Multiple inheritance ambiguity problem:
 - If class A has int data and class B also has int data, and class C inherits from both, then `obj.data` is ambiguous
 - The object of C contains A's data (bytes 200–203) and B's data (bytes 204–207) as separate memory locations
 - Resolution: use the scope resolution operator — `obj.A::data` refers to A's copy, `obj.B::data` refers to B's copy
 - Non-duplicated members (e.g. `num` in B only, `value` in C only) have no ambiguity and are accessed directly

- The deeper problem is with functions (especially virtual functions), not data members; this is why most languages restrict multiple inheritance for classes
- Object-based languages (e.g. Visual Basic): support abstraction, encapsulation, hiding, polymorphism, and modularity, but not inheritance; therefore they are not OOP languages
 - Consequence: function overriding is also unavailable (it requires inheritance), so only static polymorphism is supported

Polymorphism

- Polymorphism: the ability of an object to behave differently based on the input provided to it
- Poly = many, morphism = forms; many forms of an entity
- Two types:

Compile-time (Static) Polymorphism — Function Overloading:

- Multiple functions with the same name but different parameter signatures
- Three valid rules for overloading (any one must be satisfied):
 - Number of parameters differs (e.g. `add(int)` vs. `add(int, int)`)
 - Type of parameters differs (e.g. `add(int)` vs. `add(float)`)
 - Sequence of parameter types differs (e.g. `add(int, float)` vs. `add(float, int)`)
- Return type alone does not qualify as a basis for overloading
- C does not support function overloading:
 - C's name mangling uses only the function name to create a unique identifier
 - Two functions named `add` with different parameters would produce the same mangled name → conflict
- C++ supports function overloading:
 - C++ name mangling uses both the function name and the parameter types
 - `add(int, float)` → mangled as `add@d1f1`; `add(float, int)` → `add@f1d1`; both are unique
 - Neither C nor C++ considers return type in name mangling
- `printf` in C uses ellipsis (...), also called variable argument list, not overloading
 - Signature: `printf(const char*, ...)`
 - The format string (`%d`, `%f`, etc.) tells `printf` the type of each argument at runtime
 - This is a separate concept from function overloading

Runtime (Dynamic) Polymorphism — Function Overriding:

- Occurs only in the context of inheritance
- The child class changes the default behavior provided by the parent class
- The parent marks a method as virtual to signal that the child is permitted to override it
- If the parent does not mark the method virtual, the child cannot override it (static behavior is enforced)

- If the parent does not mark it virtual but the child does, overriding is available from that child downward, not from the original parent
- Marriage analogy:
 - Static: father insists on arranged marriage; child has no choice (behavior does not change based on input)
 - Dynamic: father leaves the choice open; child can continue with arranged marriage or choose love marriage (behavior changes based on the child's situation)
- Vivek vs. Rohit analogy:
 - Vivek gives the same response regardless of who asks → static polymorphism
 - Rohit's response changes depending on who asks → dynamic polymorphism
- In interviews: define polymorphism, name both types, then focus on dynamic polymorphism; static polymorphism is less interesting to interviewers

Modularity

- Modularity: the ability to break down a system into independent modules that interact with each other
- Two application architectures:
 - Monolithic: single tightly coupled unit; if one part fails, the whole must be replaced (e.g. a hard disk with layered platters — one bad layer means replacing the whole disk)
 - Component-based: independent, interchangeable components (e.g. a desktop PC — replace only the faulty power supply, not the whole machine)
- Advantages of modularity:
 - Components can be replaced at runtime without affecting others
 - Same component can be reused across multiple applications (reusability)
 - Example: an Intel power supply works in Lenovo, Dell, and other compatible laptops
- Code-level example:
 - Car class (main module) contains Engine module, Gearbox module, AC module, Seat module
 - Each module is independent but works in association with the others
 - An engine from one car can be swapped into another compatible car
- Definition: modularity is the ability to break down code into independent modules that interact with each other

OOP: 3 Minor Features

Persistence and Serialization

- Persistence: the ability to maintain the current state (data) of an object so it can be retrieved at a later stage
 - Example: writing data to a file before a power cut so it can be recovered later

- CSV format is an example of persistence: values stored without metadata (e.g. 1, Pritish, Java, 9999999)
- Serialization: an advancement of persistence; stores both the data and its metadata (field names, types, structure)
 - Example: XML format stores data with tags (e.g. 1, Pritish)
 - Serialization tells you not just the value but what that value represents
- Distinction: persistence stores only data values; serialization stores data values plus descriptive metadata

Concurrency

- Concurrency: the ability of an object to perform multiple tasks simultaneously at any given instance
- Real-world example: a student in this lecture is simultaneously listening, watching, writing, breathing, and their heart is beating — all concurrent behaviors
- Achieved in programming via multithreading
- C supports concurrency via `process.h` and the `_beginthreadEx` function (out of syllabus, for awareness only)
- Windows OS (written in C) supports multithreading, demonstrating that the language can support concurrency even if it is not a built-in OOP feature

RTTI and RTCI

- RTTI (Runtime Type Identification): the ability to determine the type of an object at runtime
 - In C++, implemented via the `typeid` operator, which internally uses virtual function tables (VPTR)
 - Example: progressively identifying a flying object → bird → green bird → parrot with red beak; each new piece of information narrows the type at runtime
- RTCI (Runtime Class Information): an advancement of RTTI
 - Provides complete metadata about a class at runtime: number of data members, their types, available methods, whether methods are virtual, etc.
- Feature support summary:
 - C++: all 8 features supported (abstraction, encapsulation, hiding, inheritance, polymorphism, modularity, persistence/serialization, concurrency, RTTI/RTCI)
 - C: no major OOP features fully supported; abstraction and modularity are partly achievable via structs and separate files; no encapsulation of functions, no inheritance, no polymorphism, no hiding (no access specifiers)

Function Hiding (Access Specifiers in Inheritance)

- Function hiding: restricting access to certain methods using access specifiers (private, protected, public)
- Applies to both data members and member functions

- Protected: the parent wants to pass a feature to the child but keep it hidden from the outside world
 - Example: a father's business skill is inherited by his child (Gautam) but not exposed to the general public
- Private: the parent does not want to pass a feature to the child at all
 - Example: a father's smoking habit is kept private; the child never sees it and does not inherit it
- The parent decides what is inherited (protected) and what is completely hidden (private)
- This is not a standalone OOP feature but works in conjunction with encapsulation and inheritance

Why Java: Platform Independence and Architecture

The Platform Compatibility Problem

- A Windows .exe (COFF format) cannot run on Linux; a Linux .bin cannot run on Windows
- Two root causes:
 1. Binary format is OS-specific (COFF for Windows; ELF for Linux); neither OS understands the other's format
 2. API sets are completely different: Windows uses CreateWindow (11 parameters); Linux uses openWindow (3 parameters); applications written for one OS cannot call the other's APIs
- Result: the same application (e.g. a calculator) must be rewritten separately for each platform

Three Architectural Generations

- Architecture 1 (App → Hardware):
 - App communicates directly with hardware; highest performance
 - Tight coupling: any hardware change requires rewriting the app
 - Example: replacing a Logitech mouse (left click = +5V) with a Zebronics mouse (left click = +1.5V) breaks the app
 - Used in embedded systems: washing machines, calculators, CRT TV games, lathe machines, laser cutters, 3D printers
- Architecture 2 (App → OS → Hardware):
 - OS introduced as an indirection layer; app no longer needs to know hardware specifics
 - Hardware changes only require OS/driver updates, not app rewrites
 - HAL (Hardware Abstraction Layer) sits between OS and hardware; device drivers reside here
 - Plug-and-play standardized device driver formats so the OS can communicate with new hardware automatically

- Disadvantage: app is now dependent on the OS; different OS = different APIs = app must be rewritten
- Architecture 3 (App → JVM → OS → Hardware):
 - A virtual machine layer introduced between app and OS
 - App is now independent of both hardware and OS
 - Performance degrades further with each added layer (more intermediaries = slower communication)
 - Analogy: “Vaishnavi, tell Pallavi to tell Pooja to unmute” vs. directly saying “Om, unmute” — more layers = more latency
 - Advantage: write the application once; it runs on any platform that has a JVM installed

James Gosling and the Origin of Java

- James Gosling worked at Sun Microsystems
- In 1990, he surveyed US households and projected that by 2000 each home would use ~200 microprocessors (washing machines, TVs, microwaves, refrigerators, etc.)
- Problem identified: Samsung sourcing microprocessors from Intel (3,000 units), AMD (5,000 units), and a third vendor (2,000 units) — three different architectures — would need to rewrite the same calculator/calendar/game logic three times
- Gosling proposed a platform-independent language: write once, run anywhere
- Originally named Oak (after a tree outside his office); renamed Java (slang for coffee) after copyright conflicts
- Java officially introduced in 1994 as the first OOP language with platform independence as its core objective
- Syntax borrowed from C; OOP principles borrowed from C++
- Sun Microsystems acquired by Oracle in 2014; Java is now an Oracle product

Java Architecture: JDK, JRE, JVM, and Bytecode

Compilation and Execution Flow

- Three types of programming languages:
 - Compiled (e.g. C, C++): entire code compiled to machine code, then executed
 - Interpreted (e.g. BASIC, scripting languages): read, compile, and execute line by line
 - Parsed (e.g. HTML): character-by-character analysis by a parser
- Java combines compiled and interpreted approaches to achieve platform independence:
 - Developer writes source code in a .java file
 - javac (Java compiler) compiles the .java file → generates .class file (bytecode)
 - Bytecode is not binary (not OS-readable) and not plain text; it is an intermediate format
 - Bytecode is only understood by the Java architecture (JVM), not by any OS directly

- One .class file is generated per class; five classes in one .java file → five .class files
 - java (JVM launcher) loads the .class bytecode into the JVM
 - JVM interprets/executes the bytecode on the underlying OS and hardware
- Bytecode is platform-independent; the JVM is platform-specific
 - The same .class file runs on Windows, Linux, or Solaris without modification
 - A different JVM is installed for each OS

JIT Compiler (Performance Improvement)

- Java 1.0 (1994): JVM interpreted bytecode line by line on every execution → very slow
- Java 2 (1998): introduced JIT (Just-In-Time) compilers
 - On first execution, JIT compiles the entire bytecode to machine code at once
 - Subsequent executions use the cached machine code → significantly faster
 - Performance still not as fast as native compiled languages, but dramatically improved

JDK vs. JRE

- Java SE (Standard Edition) bifurcates into two components:
 - JDK (Java Development Kit): all tools, libraries, and debugger needed to develop, execute, and debug Java applications; includes the JRE
 - JRE (Java Runtime Environment): tools and libraries needed only to execute (run) a Java application; no compiler or debugger
- Rule: install JDK to write and develop Java programs; install JRE only to run pre-built Java programs
- Both JDK and JRE are OS-specific; a different version exists for Windows, Linux, Solaris, etc.
- Translator analogy: Shivam traveling to China needs a Chinese translator; to Japan, a Japanese translator; the English input (bytecode) stays the same, but the translator (JVM) changes per destination (OS)

Java SE Editions

- J2SE → JSE (Java Standard Edition): core language fundamentals, desktop apps, networking, file I/O, JDBC (database connectivity)
- J2EE → JEE (Java Enterprise Edition): distributed / N-tier architecture (e.g. Amazon — front-end in India, product catalog in China, payment gateway in Australia, credit card DB in the US)
- J2ME → JME (Java Micro Edition): embedded and handheld devices (PDAs, set-top boxes, Blu-ray players, early mobile phones)
- Current industry versions: JDK 8 and JDK 11 most widely used; CDAC syllabus incorporates some JDK 17 features; latest available is JDK 25/26

Why Not Just Rewrite Apps Per Platform?

- 32+ years of Java code exists (trillions of programs); only a few hundred new OS platforms have emerged in that time
- Writing a new JVM for a new platform is far more practical than converting all existing Java code to that platform
- Only the OS-interfacing portion of the JVM needs to be rewritten for a new platform; the bytecode-reading portion stays the same

First Java Program: Hello World

100% OOP Rules (Java is close but not fully compliant)

- Three rules for a 100% OOP language:
 1. Everything must be an object or treated as an object
 2. Nothing can be global (including the main function)
 3. All OOP features must be supported
- Java is close to 100% OOP but not fully, due to primitive data types (int, float, etc.) which are not objects

Writing and Running Hello World

- Because nothing can be global, even main must live inside a class:

```
class Program { public static void main(String[] args) { System.out.println("Hello World"); } }
```

- main signature breakdown:
 - public: accessible from anywhere (required for JVM to call it)
 - static: callable at the class level without creating an object
 - void: returns nothing
 - String[] args: command-line arguments (not used here)
 - This exact signature is mandatory; only the position of String[] args is flexible
- System.out.println:
 - System is a class; out is an object inside it; println is a method on out
 - Equivalent to printf in C; out is encapsulated inside System because globals are not allowed
 - println adds a newline after printing; printf is also available as System.out.printf
- File naming rule:
 - If the class is declared public, the filename must exactly match the class name (e.g. Program.java)
 - If not public, any filename is valid (e.g. Anything.java with class Program inside)
- Compilation and execution steps:
 - Set PATH: set PATH=C:\JavaSE\bin (tells OS where to find javac.exe and java.exe)

- Navigate to file location: `cd E:\Kharghar`
- Compile: `javac Anything.java` (or `javac *.java`)
- If successful, `Program.class` is generated in the same directory
- Run: `java Program` (specify the class name, not the file name)
- Java home folder: the directory containing `bin` and `lib` folders (e.g. `C:\JavaSE`)
 - `javac.exe`: the Java compiler
 - `java.exe`: the JVM launcher
 - The `bin` folder also contains platform-specific DLLs (Windows-specific)

Action Items

- **Install JDK 21 on local machine** Download from Oracle (search "Java download"); install to a short path like `C:\JavaSE` to make command-line navigation easier. Both MSI and ZIP installers are fine.
- **Write and run the Hello World program from the command prompt** Code shared in chat; set `PATH` to the `bin` folder, compile with `javac`, run with `java`, and confirm the `.class` bytecode file is generated alongside the `.java` file.
- **Revise today's concepts before tomorrow's 9 AM lecture** Covers compiler pipeline, static vs. dynamic libraries, OOP features, Java architecture, and the Hello World walkthrough; instructor will share screen recordings and notes.
- **Decide on a team name** Vipul sir will ask for team names within the next day or two; agree on a name with teammates before then.

Chat with meeting transcript:

<https://notes.granola.ai/t/d17758fa-9cd4-4a19-b777-3ec8a6cef7a8-00demib2>